**Unidad: 2 Actividad entregable: Diagnóstico de vulnerabilidades: análisis y evaluación de seguridad en software.**

Diana Zoraida Romero Ramírez

ID: 949490.

Corporación Universitaria Minuto de Dios UNIMINUTO

Programa: Ingeniería de Sistemas

NRC: 40-63386 Desarrollo de software seguro.

Docente: Edwin Albeiro Ramos Villamil.

Abril, 19, 2025

Bogotá D.C

## 1. Introducción

En el desarrollo de software, la seguridad y la calidad del código son aspectos fundamentales para garantizar que una aplicación funcione correctamente, sea confiable y pueda mantenerse en el tiempo. Por esta razón, actualmente no solo es importante que un sistema cumpla con sus funcionalidades, sino también que su estructura interna sea revisada y evaluada mediante herramientas especializadas que permitan identificar errores, vulnerabilidades y malas prácticas de programación.

El presente informe tiene como finalidad documentar el análisis realizado a un software web desarrollado en PHP con conexión a base de datos MySQL, el cual fue ejecutado localmente para comprobar su funcionamiento y posteriormente evaluado mediante SonarQube Cloud. A partir de este proceso, se identificaron diferentes hallazgos relacionados con seguridad, fiabilidad y mantenibilidad del código fuente.

Con base en los resultados obtenidos, se realizó la revisión de los errores detectados, el análisis de sus posibles causas y la propuesta de corrección dentro del código, con el propósito de mejorar la calidad del software y evidenciar un proceso de diagnóstico y solución sustentado técnicamente. De esta manera, el informe presenta de forma ordenada cada una de las etapas desarrolladas, desde la preparación del entorno y la ejecución del sistema, hasta la identificación, corrección y validación final de los problemas encontrados.

## 2. Objetivos

### 2.1 Objetivo general

Analizar la calidad y seguridad de un software web mediante el uso de la herramienta SonarQube Cloud, con el fin de identificar errores, vulnerabilidades y malas prácticas en el código fuente, y posteriormente proponer e implementar soluciones que permitan mejorar su funcionamiento y mantenibilidad.

### 2.2 Objetivos específicos

- Ejecutar el software en un entorno local para verificar su correcto funcionamiento y conexión con la base de datos.
- Configurar e integrar el proyecto en SonarQube Cloud para realizar el análisis automático del código fuente.
- Identificar los diferentes tipos de problemas detectados por la herramienta, como errores de seguridad, fallos de fiabilidad y problemas de mantenibilidad.
- Analizar las causas de los errores encontrados en el código del software.
- Realizar la corrección de los errores identificados, aplicando buenas prácticas de programación.
- Validar nuevamente el software después de las correcciones para comprobar la mejora en la calidad del código.

## 3. Descripción del software analizado

El software analizado corresponde a una aplicación web desarrollada en PHP bajo una arquitectura tipo MVC (Modelo-Vista-Controlador), la cual permite la gestión de información mediante una interfaz accesible desde el navegador. Este sistema se encuentra conectado a una base de datos MySQL, lo que le permite almacenar, consultar y manipular datos de manera estructurada.

La aplicación tiene como objetivo principal la administración de un sistema tipo inventario, en el cual se pueden gestionar usuarios, productos y otros elementos relacionados. A través de sus funcionalidades, el usuario puede interactuar con el sistema

mediante formularios, realizar consultas y ejecutar diferentes operaciones propias de un entorno de gestión.

El software fue ejecutado en un entorno local utilizando XAMPP, lo que permitió levantar el servidor Apache y establecer la conexión con la base de datos previamente configurada. Durante esta fase, se verificó el correcto funcionamiento del sistema, incluyendo el acceso mediante credenciales y la interacción con las diferentes funcionalidades disponibles.

Desde el punto de vista técnico, el sistema presenta una estructura organizada en carpetas como controladores, modelos y vistas, lo cual facilita la separación de responsabilidades dentro del código. Sin embargo, como parte del proceso de análisis, se identificó la necesidad de evaluar la calidad del código fuente, con el fin de detectar posibles errores, vulnerabilidades o malas prácticas de desarrollo.

Por esta razón, el software fue posteriormente integrado con la herramienta SonarQube Cloud, permitiendo realizar un análisis estático del código para identificar problemas relacionados con seguridad, fiabilidad y mantenibilidad, los cuales serán detallados en las siguientes secciones del informe.

## 4. Metodología / Procedimiento

Para el desarrollo del presente análisis se llevó a cabo una serie de pasos organizados que permitieron ejecutar el software, verificar su funcionamiento y posteriormente evaluar la calidad del código mediante el uso de SonarQube Cloud. A continuación, se describe el procedimiento realizado:

### 1. Preparación del entorno de trabajo

Inicialmente, se instaló y configuró el entorno de desarrollo local utilizando XAMPP, el cual permitió disponer de los servicios necesarios como Apache y MySQL. Asimismo, se utilizó Visual Studio Code como editor de código para la revisión y modificación de los archivos del proyecto.

### 2. Configuración y ejecución del software

El software fue descomprimido y ubicado dentro de la carpeta htdocs de XAMPP, con el fin de poder ejecutarlo desde el navegador mediante un servidor local. Posteriormente, se creó la base de datos en phpMyAdmin a partir del archivo proporcionado en el proyecto.

Se configuró la conexión a la base de datos dentro del archivo correspondiente, ajustando parámetros como el puerto (3307), el usuario y la base de datos, lo que permitió establecer correctamente la comunicación entre la aplicación y MySQL.

Finalmente, se verificó el correcto funcionamiento del sistema accediendo desde el navegador y realizando pruebas de inicio de sesión e interacción con las funcionalidades disponibles.

**3. Control de versiones con GitHub**

Una vez validado el funcionamiento del software, se procedió a crear un repositorio en GitHub, donde se subió el proyecto utilizando Git desde Visual Studio Code. Esto permitió gestionar el código y facilitar la integración con herramientas externas de análisis.

**4. Integración con SonarQube Cloud**

Posteriormente, se accedió a SonarQube Cloud mediante autenticación con GitHub, autorizando el acceso al repositorio creado. Se configuró una organización dentro de la plataforma y se seleccionó el proyecto a analizar.

Se realizó la configuración inicial del análisis, definiendo los parámetros básicos del proyecto y estableciendo una definición de código basada en un rango de días.

**5. Ejecución del análisis de código**

Una vez configurado el proyecto en la plataforma SonarQube Cloud, se procedió a ejecutar el análisis automático del código fuente del software. Este proceso permitió evaluar diferentes aspectos del sistema, tales como la seguridad, la fiabilidad y la mantenibilidad.

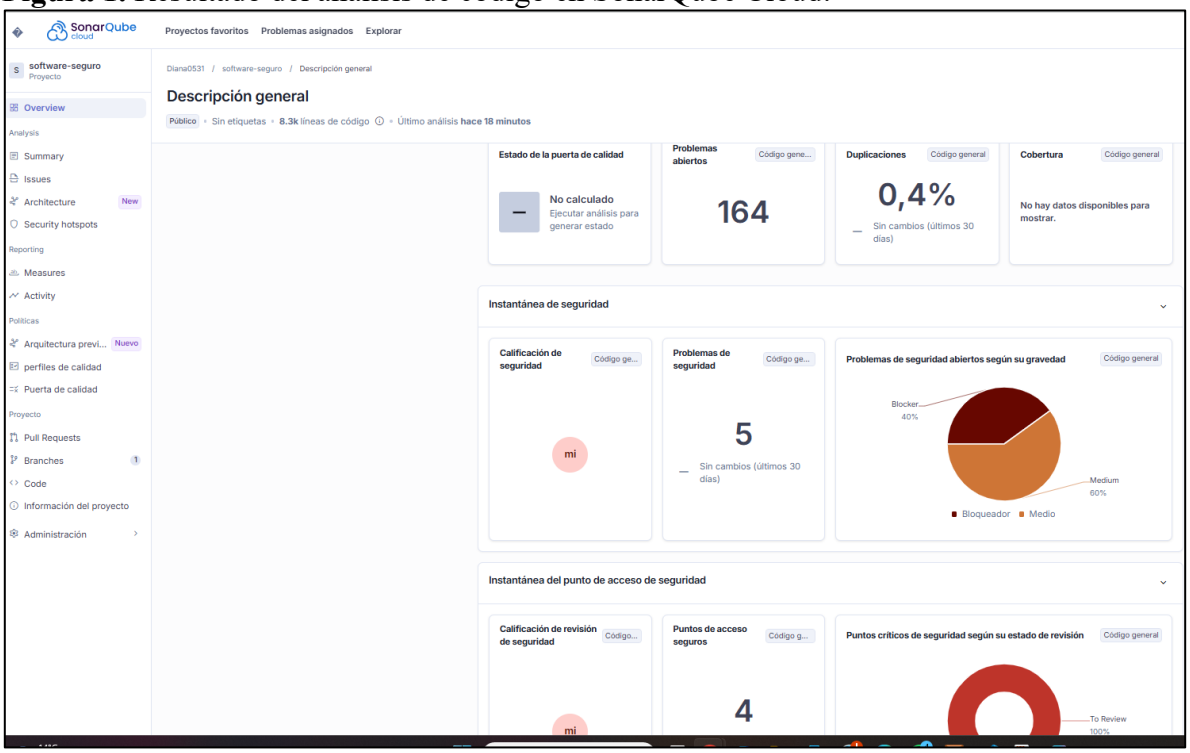
Como resultado del análisis, la herramienta generó un resumen general en el cual se evidencian múltiples problemas en el código, clasificándolos según su nivel de gravedad e

impacto. En total, se identificaron 164 problemas abiertos, lo que indica la presencia de diversas oportunidades de mejora en el software.

Asimismo, se observaron indicadores relevantes como una calificación de seguridad baja, con 5 problemas de seguridad detectados, además de una calificación de fiabilidad y mantenibilidad que reflejan la necesidad de realizar ajustes en el código. También se evidenció un bajo porcentaje de duplicación de código, lo cual es un aspecto positivo dentro del análisis.

La siguiente figura muestra el resultado general obtenido tras la ejecución del análisis en SonarQube Cloud:

**Figura 1.** Resultado del análisis de código en SonarQube Cloud.




*Nota:* Fuente propia.

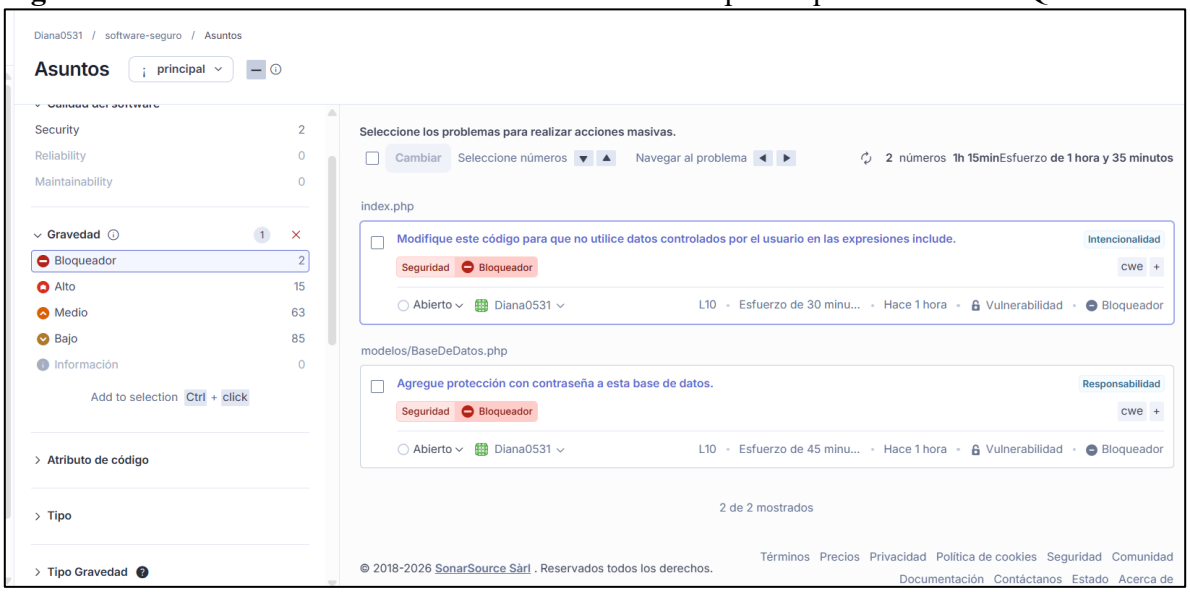
## 5. Análisis de errores detectados

Una vez realizado el análisis del software mediante la herramienta SonarQube Cloud, se procedió a revisar en detalle los problemas identificados en la sección de “Issues”, donde se listan los errores, advertencias y vulnerabilidades presentes en el código fuente.

Estos problemas se encuentran clasificados según su tipo y nivel de severidad, incluyendo aspectos relacionados con la seguridad, la fiabilidad y la mantenibilidad del sistema. A partir de este análisis, se seleccionaron algunos de los errores más relevantes para su estudio y posterior corrección.

A continuación, se presenta el análisis de uno de los errores identificados:

**Figura 2.** Identificación de vulnerabilidades críticas de tipo bloqueador en SonarQube Cloud



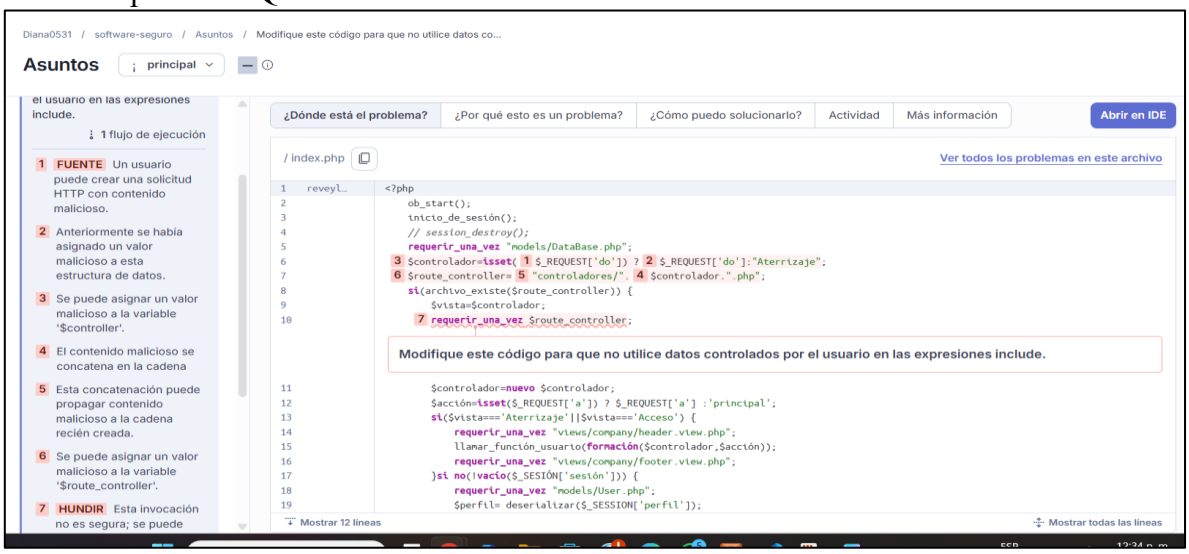

*Nota:* Fuente propia.

**Análisis del error 1: Análisis y corrección de vulnerabilidad en index.php**

Durante el análisis del código mediante SonarQube Cloud, se identificó una vulnerabilidad crítica en el archivo index.php, clasificada como de tipo bloqueador. El problema se presenta porque la aplicación utiliza datos controlados por el usuario mediante $_REQUEST['c'] para construir dinámicamente la ruta de un archivo que posteriormente es incluido con require_once, sin realizar una validación previa.

Esta práctica representa un riesgo, ya que un atacante podría manipular el valor del parámetro y forzar la inclusión de archivos no autorizados dentro del sistema. Este tipo de vulnerabilidad está asociado a la inclusión dinámica insegura de archivos, lo que puede comprometer la seguridad de la aplicación. Debido a su impacto, SonarQube lo clasifica como un problema de alta gravedad que debe ser corregido de manera prioritaria.

**Figura 3.** Vulnerabilidad por uso de datos controlados por el usuario en el archivo index.php detectada por SonarQube Cloud




*Nota:* Fuente propia.

## Corrección implementada

Para mitigar la vulnerabilidad identificada, se implementó una validación del controlador solicitado. En lugar de utilizar directamente el valor proporcionado por el usuario, se obtuvo una lista de controladores válidos a partir de los archivos existentes en la carpeta controllers.

Posteriormente, se verificó que el controlador solicitado se encuentre dentro de dicha lista. En caso contrario, el sistema asigna un controlador por defecto (Landing). Esta solución permite mantener el funcionamiento original del sistema sin afectar procesos como el inicio de sesión, al tiempo que mejora significativamente la seguridad del código.

**Figura 4.** Código corregido implementando validación de controladores permitidos.

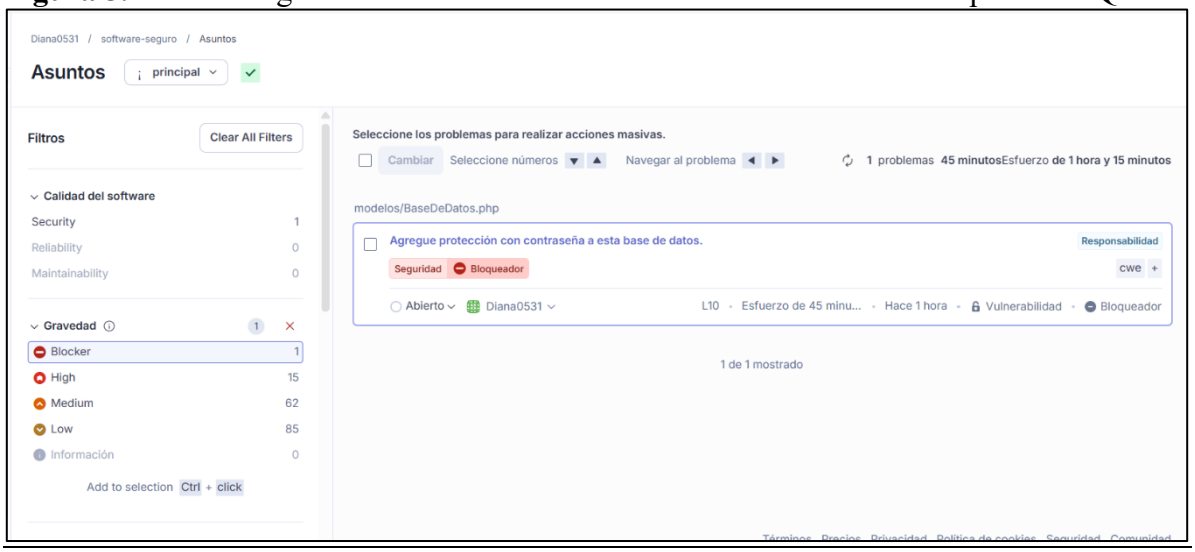
```php
$availableControllers = array_map(function ($file) {
    return basename($file, ".php");
}, glob("controllers/*.php"));

$requestedController = isset($_REQUEST['c']) ? $_REQUEST['c'] : $defaultController;

if (!in_array($requestedController, $availableControllers, true)) {
    $requestedController = $defaultController;
}
```

*Nota:* Fuente propia.

**Resultado de la corrección:**

Después de aplicar la solución y ejecutar nuevamente el análisis en SonarQube Cloud, se evidenció la eliminación de la vulnerabilidad en el archivo index.php. El número de problemas críticos disminuyó, lo que confirma que la corrección implementada fue efectiva. Además, el sistema continuó funcionando correctamente, permitiendo la navegación y el inicio de sesión sin inconvenientes.

**Figura 5.** Error de seguridad en la conexión a la base de datos identificado por SonarQube




*Nota:* Fuente propia.

**Análisis del error 2: Exposición de credenciales en DataBase.php**

Durante el análisis del software mediante SonarQube Cloud, se identificó una vulnerabilidad de seguridad en el archivo DataBase.php, relacionada con el manejo inseguro de las credenciales de acceso a la base de datos. Inicialmente, el sistema realizaba la conexión utilizando el usuario root y una contraseña vacía, lo cual representaba un riesgo crítico para la seguridad del aplicativo y de la información almacenada.

Este hallazgo fue clasificado por SonarQube como un problema de tipo bloqueador, debido a que una base de datos sin contraseña o con credenciales expuestas puede permitir accesos no autorizados, modificación o eliminación de datos, robo de información sensible y afectación de la integridad del sistema. En otras palabras, cualquier persona que lograra conectarse al servidor podría manipular el contenido de la base de datos sin controles mínimos de autenticación.

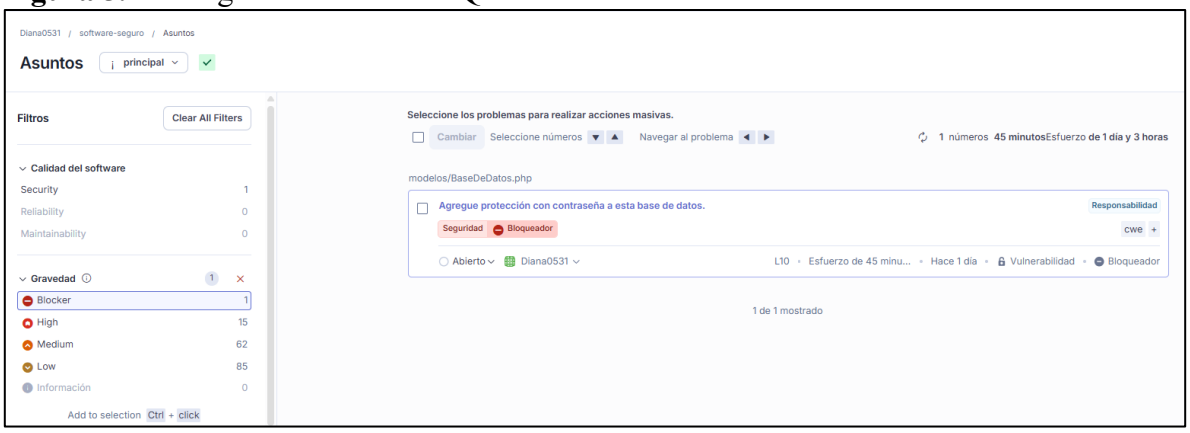
Posteriormente, se realizó una primera corrección asignando una contraseña al usuario de la base de datos. Sin embargo, SonarQube detectó que dicha contraseña había quedado escrita directamente dentro del código fuente, lo cual también constituye una vulnerabilidad, ya que las credenciales no deben quedar codificadas de manera fija dentro de la aplicación. Esto se debe a que, si el código es compartido, almacenado en un repositorio o revisado por terceros, la contraseña queda expuesta y puede ser reutilizada de manera maliciosa.

Por esta razón, la solución definitiva consistió en reemplazar la contraseña escrita directamente en el código por una variable de entorno, permitiendo que la credencial sea obtenida desde una fuente externa más segura. Con ello, se logró eliminar la exposición directa de la contraseña y mejorar el manejo seguro de la configuración de conexión.

**Evidencia inicial del error**

En la primera revisión realizada en SonarQube Cloud, se observó el problema de seguridad relacionado con la ausencia de protección mediante contraseña en la base de datos.

**Figura 5.** Hallazgo inicial en SonarQube Cloud: conexión a la base de datos sin contraseña.




*Nota:* Fuente propia.

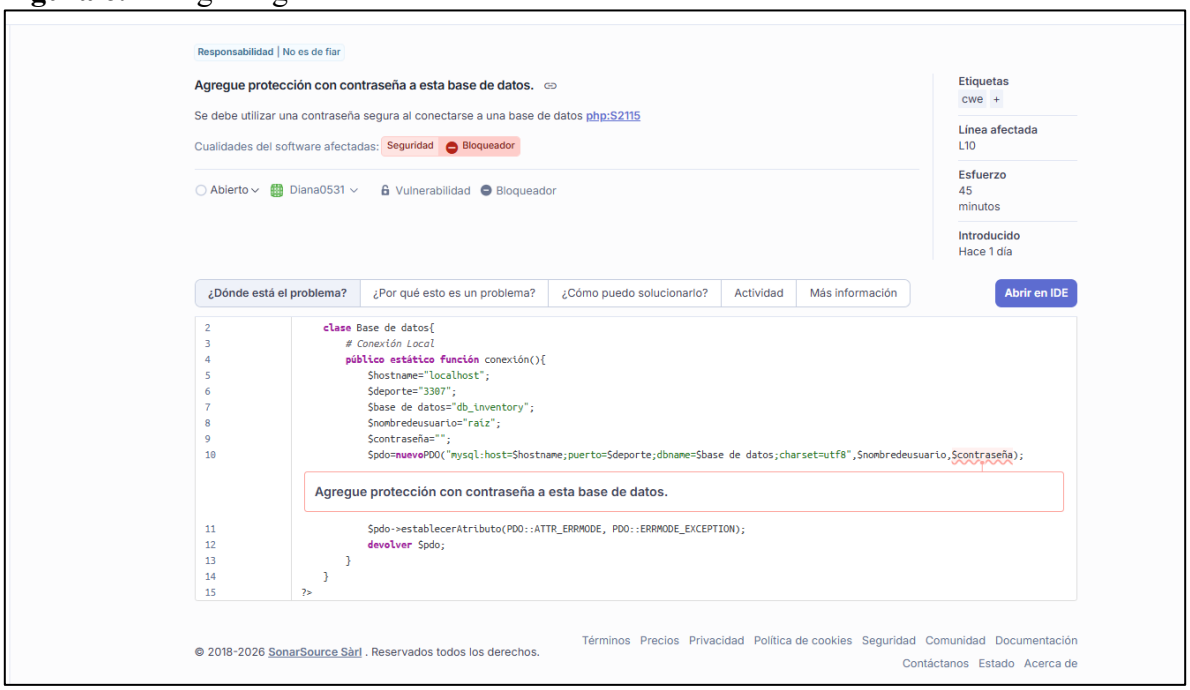
**Código vulnerable inicial**

El fragmento de código originalmente identificado como vulnerable era el siguiente:

```
$username = "root";

$password = "";
```

En este caso, la conexión se realizaba con el usuario administrador y sin ninguna contraseña, lo que representaba una configuración insegura.

**Figura 6.** Código original con conexión a base de datos sin contraseña.




*Nota:* Fuente propia.

## Primera corrección aplicada

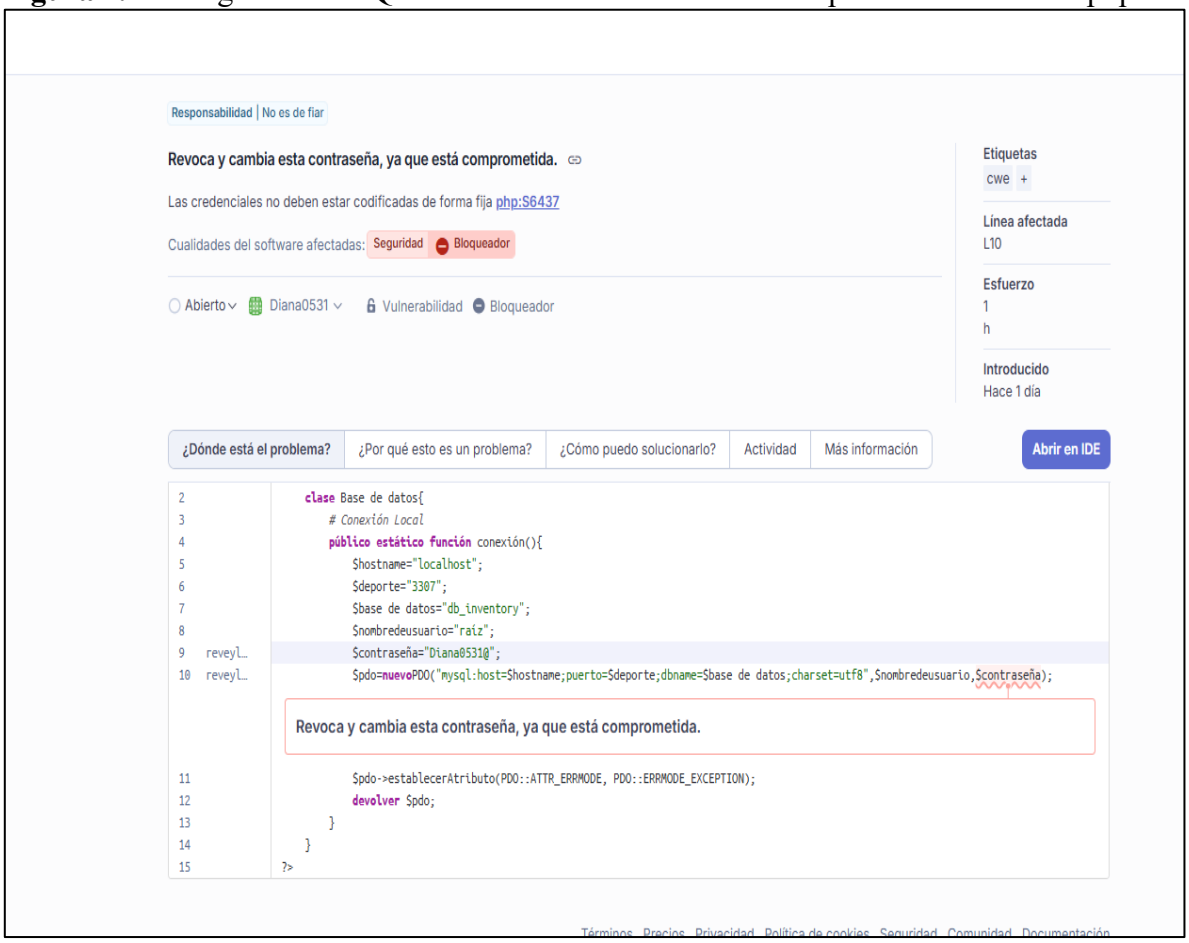
Como primera medida, se configuró una contraseña segura para el usuario de MySQL y se actualizó la conexión del sistema para permitir nuevamente el acceso a la base de datos. Esta corrección resolvió el problema de conexión sin contraseña, pero generó un nuevo hallazgo en SonarQube debido a que la clave quedó escrita directamente en el archivo DataBase.php.

Este nuevo escenario permitió evidenciar que no solo es importante usar contraseña, sino también gestionarla adecuadamente para evitar su exposición dentro del código fuente.

## Segundo hallazgo detectado

Después de incorporar la contraseña, SonarQube reportó una nueva advertencia indicando que la credencial estaba comprometida por encontrarse codificada directamente en el archivo de conexión.

**Figura 7.** Hallazgo de SonarQube relacionado con credencial expuesta en DataBase.php.




*Nota:* Fuente propia.

## Corrección final implementada

Para corregir de forma adecuada esta vulnerabilidad, se reemplazó la contraseña fija por una variable de entorno, utilizando la función getenv(). De esta manera, la aplicación obtiene la credencial desde una configuración externa, evitando dejar información sensible visible en el código.

El fragmento corregido quedó así:

$password = getenv('MYSQL_SECURE_PASSWORD') ?: "Diana0531@";

Esta modificación permitió mejorar la seguridad del archivo de conexión, ya que separa la configuración sensible del código principal y reduce la exposición de credenciales dentro del proyecto.

**Figura 8.** Código corregido en DataBase.php utilizando variable de entorno para la contraseña.

```php
<?php
    class DataBase{
        #  Conexión Local
        public static function connection(){
            $hostname = "localhost";
            $port = "3307";
            $database = "db_inventory";
            $username = "root";
            $password = getenv('MYSQL_SECURE_PASSWORD') ?: "Diana0531@";
            $pdo = new PDO("mysql:host=$hostname;port=$port;dbname=$database;charset=utf8",$username,$password);
            $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
            return $pdo;
        }
    }
?>
```

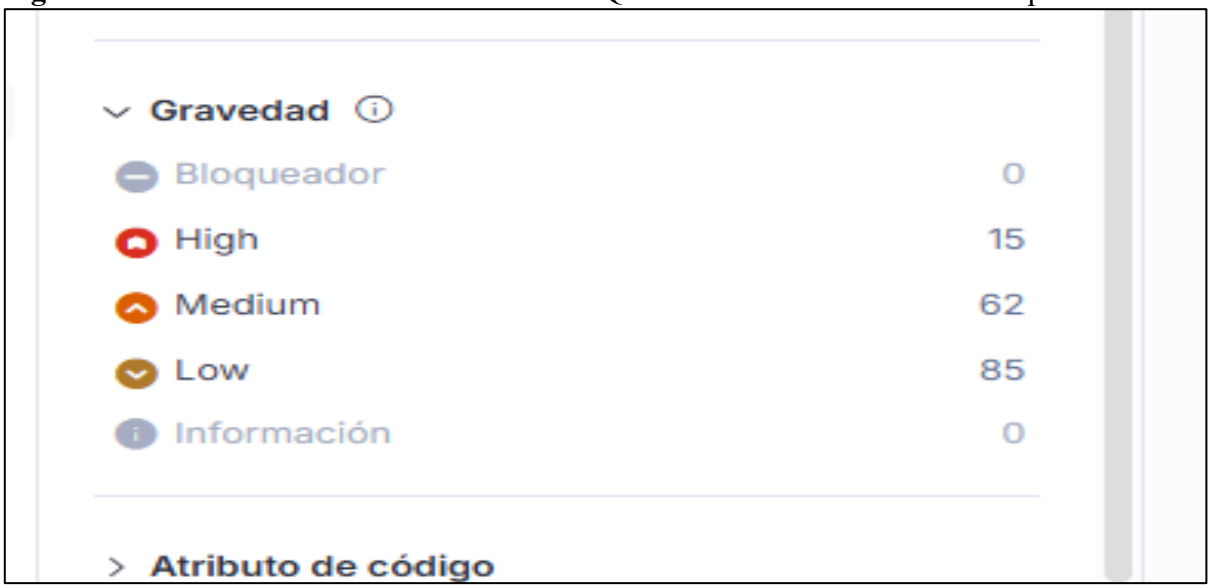
*Nota:* Fuente propia.

**Validación de la corrección**

Una vez aplicada la corrección y subidos los cambios al repositorio, se ejecutó nuevamente el análisis en SonarQube Cloud. Como resultado, las vulnerabilidades clasificadas como bloqueadoras fueron eliminadas, evidenciando que el problema fue resuelto correctamente.

Esto confirma que la aplicación pasó de tener hallazgos críticos de seguridad a no presentar problemas de nivel bloqueador, lo cual representa una mejora importante en la calidad y seguridad del software.

**Figura 9.** Resultado final del análisis en SonarQube Cloud con cero errores bloqueadores.




*Nota:* Fuente propia.

**Interpretación del resultado**

La corrección aplicada permitió fortalecer la seguridad del software en dos niveles. En primer lugar, se evitó el acceso a la base de datos sin autenticación, estableciendo una contraseña para el usuario correspondiente. En segundo lugar, se mejoró el manejo de credenciales al evitar que la contraseña quedara escrita directamente en el código fuente.

Este proceso demuestra que una vulnerabilidad puede requerir varias etapas de corrección: primero resolver el riesgo inmediato y luego ajustar la implementación para cumplir con buenas prácticas de desarrollo seguro. Además, evidencia la importancia de herramientas como SonarQube Cloud para identificar problemas que no siempre son visibles a simple vista durante la programación.

En conclusión el análisis del archivo DataBase.php permitió identificar y corregir una vulnerabilidad crítica asociada al manejo inseguro de credenciales de base de datos. Inicialmente, el sistema presentaba una conexión sin contraseña, lo que representaba un riesgo severo. Posteriormente, aunque se incorporó una clave, fue necesario aplicar una mejora adicional para evitar su exposición directa en el código. Finalmente, mediante el uso de variables de entorno, se logró una solución más segura y alineada con buenas prácticas, validada posteriormente en SonarQube Cloud al obtener un resultado sin errores bloqueadores.

## 6. Resultados posteriores a la corrección de vulnerabilidades

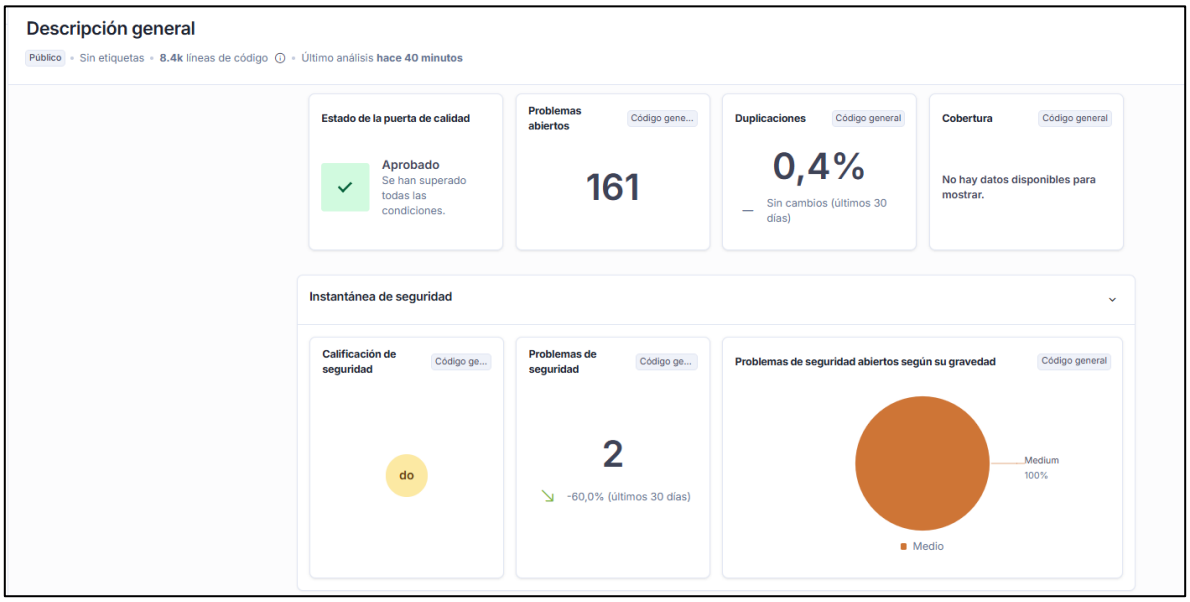
Una vez aplicadas las correcciones correspondientes a las vulnerabilidades identificadas en el sistema, se realizó un nuevo análisis mediante la herramienta SonarQube Cloud con el fin de evaluar el impacto de las mejoras implementadas.

Como resultado, se evidenció una disminución significativa en la cantidad de problemas críticos del software, especialmente aquellos clasificados como bloqueadores, los cuales fueron completamente eliminados. Adicionalmente, se observa que el estado de la puerta de calidad se encuentra en estado “Aprobado”, lo que indica que el sistema cumple con los criterios mínimos establecidos en términos de seguridad y calidad del código.

Asimismo, aunque aún existen algunos problemas de menor gravedad (niveles medio, alto y bajo), estos no representan un riesgo crítico para el funcionamiento del sistema, pero sí constituyen oportunidades de mejora continua en el desarrollo del software.

Este resultado confirma que las acciones correctivas implementadas fueron efectivas, permitiendo fortalecer la seguridad del sistema, mejorar la calidad del código y reducir los riesgos asociados a vulnerabilidades previamente detectadas.

**Figura 10.** Resultados del análisis en SonarQube Cloud después de la corrección de vulnerabilidades




*Nota:* Fuente propia.

Se observa la mejora en la calidad del software tras la implementación de las correcciones, evidenciando la eliminación de errores críticos y la aprobación de la puerta de calidad en SonarQube Cloud.

## 7. Conclusión

A lo largo de este trabajo se pudo evidenciar la importancia de analizar el código fuente utilizando herramientas como SonarQube Cloud, ya que permitió identificar vulnerabilidades que a simple vista no se detectan durante el desarrollo. En este caso, se encontraron errores críticos relacionados con el uso de datos proporcionados por el usuario sin validación y el manejo inseguro de las credenciales de acceso a la base de datos.

Durante el proceso, no solo se identificaron los problemas, sino que también se comprendió el impacto que estos pueden tener en la seguridad del sistema, como accesos no autorizados, manipulación de información y posibles ataques. A partir de esto, se aplicaron soluciones adecuadas, como la validación de datos y el uso de variables de entorno para proteger la información sensible.

Finalmente, al ejecutar nuevamente el análisis, se observó una mejora significativa en la calidad del software, evidenciada en la eliminación de errores de tipo bloqueador y en la aprobación de la puerta de calidad. Esto demuestra que aplicar buenas prácticas de desarrollo seguro no solo mejora el funcionamiento del sistema, sino que también reduce riesgos y fortalece la confianza en la aplicación.

En conclusión, este ejercicio permitió reforzar conocimientos sobre seguridad en el desarrollo de software y evidenció la importancia de implementar medidas preventivas desde las primeras etapas del desarrollo.

## 8. Referencias

41 common web application vulnerabilities explained - SecurityScoreCard. (2026, 18 marzo). SecurityScorecard. https://securityscorecard.com/blog/common-web-application-vulnerabilities-explained/

Las 10 amenazas principales de aplicaciones web de OWASP. (s. f.). Apigee Docs. https://docs.apigee.com/api-platform/faq/owasp-protection?hl=es-419

Rapid. (s. f.). Web Application vulnerabilities | SQLI, XSS & CSRF - Rapid7. Rapid7. https://www.rapid7.com/fundamentals/web-application-vulnerabilities/

Sillam, Y. (s. f.). What is PHP Injection | Types, Examples & Prevention | Imperva. Learning Center. https://www.imperva.com/learn/application-security/php-injection/

Thales, G. (2025, 10 noviembre). ¿Qué es la Seguridad del Software y por qué es tan Importante Hoy? Thales Cloud Security Products. https://cpl.thalesgroup.com/es/software-monetization/

what-is-software-security Using Environment Variables for Authentication Credentials | Sauce Labs Documentation. (2025, 26 mayo). https://docs.saucelabs.com/basics/environment-variables/

Virginia.Cordero, & Virginia.Cordero. (2026, 12 marzo). SonarQube: ¿qué es y para qué sirve? https://apiquality.io/es/sonarqube-que-es-y-para-que-sirve/